

Practical Exploration of Snort IDS with Custom Rule Development and Wazuh SIEM Integration for Network Security Monitoring

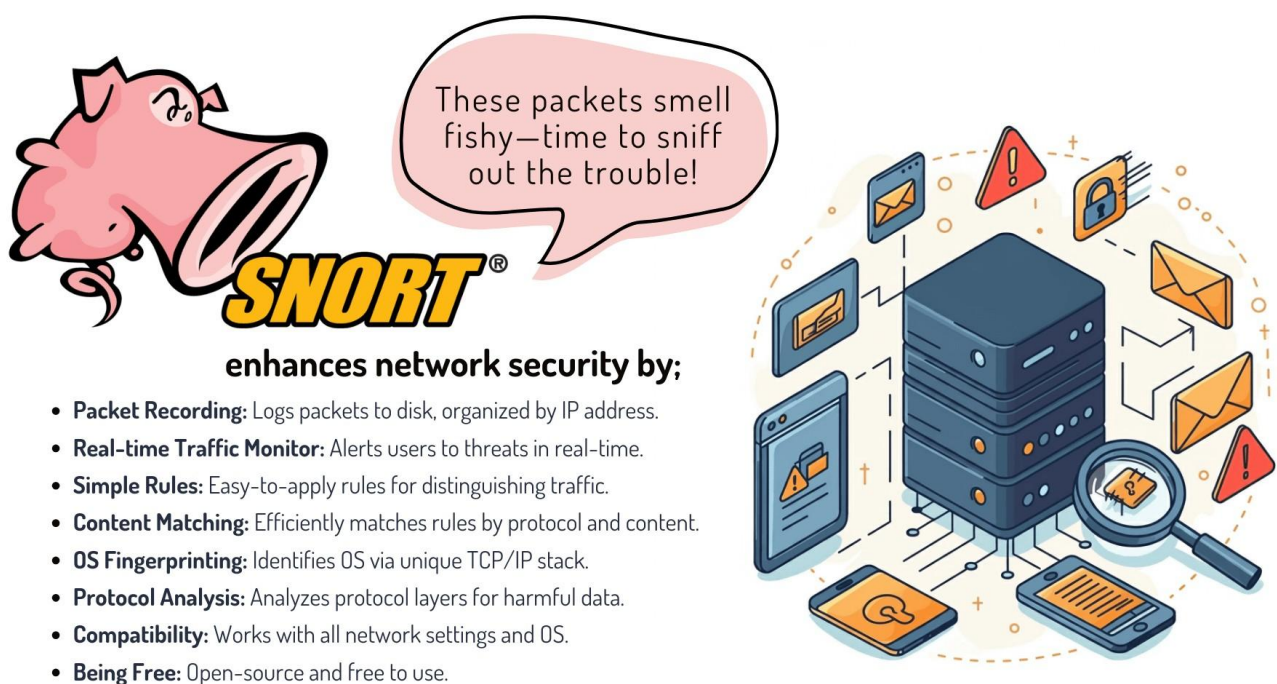
Project Overview

This project focuses on **hands-on exploration of Snort**, an open-source **Network Intrusion Detection System (NIDS)**, by installing, configuring, and actively testing it in a real lab environment.

The goal was not just installation, but **practical learning**, including:

- Understanding how Snort inspects network traffic
- Creating and testing **custom detection rules**
- Using **Snorpy (GUI-based rule creator)** to simplify rule development
- Simulating attacker activity from a Kali Linux machine
- Integrating Snort with **Wazuh SIEM** for centralized log collection, monitoring, and analysis

This project demonstrates how **network-level detections** can be correlated and visualized within a SIEM platform, enabling security analysts to monitor network threats from a single interface.



wazuh.

Tools & Technologies Used

- **Snort (v2.9.x)** – Network Intrusion Detection System
- **Snorpy** – Web-based Snort rule creation tool
- **Wazuh Agent & Manager** – SIEM and log correlation platform
- **Ubuntu (AWS EC2)** – Snort & Wazuh Agent host
- **Kali Linux** – Attacker simulation machine
- **FTP / ICMP / TCP traffic** – For rule testing
- **AWS EC2 Networking** – Cloud-based lab environment

Why Snort?

Snort is a powerful IDS because it can:

- Inspect live network traffic in real time
- Detect malicious behavior using **signature-based rules**
- Be customized for environment-specific threats
- Work efficiently in both on-premise and cloud environments

By integrating Snort with Wazuh, network alerts can be:

- Centralized
- Correlated
- Visualized
- Investigated efficiently by SOC analysts

Architecture Overview

Lab Architecture:

Kali Linux (Attacker)

|
| ICMP / FTP / TCP Traffic
|

Ubuntu EC2 (Snort + Wazuh Agent)

|
| Snort Alerts (fast.alert logs)
|

Wazuh Manager (SIEM Dashboard)

Implementation Steps

Step 1: Installing Required Dependencies for Snort

Installed all required libraries and build dependencies to ensure Snort functions correctly for packet capture, rule parsing, and protocol analysis.

Purpose:

To prepare the system with all necessary components required by Snort.

```
root@client:/home/ubuntu# sudo apt install -y \
build-essential \
libpcap-dev \
libpcre3-dev \
libdumbnet-dev \
zlib1g-dev \
liblzma-dev \
openssl \
libssl-dev
```

Step 2: Installing and Configuring Snort

Snort was installed using the package manager and configured to monitor the **local network range**.

- Defined HOME_NET as the EC2 private subnet
- Configured Snort to inspect traffic relevant to the host

Purpose:

To tell Snort *which network traffic is considered internal* and should be monitored

Configuring snort

Please use the CIDR form - for example, 192.168.1.0/24 for a block of 256 addresses or 192.168.1.42/32 for just one. Multiple values should be comma-separated (without spaces).

You can leave this value empty and configure HOME_NET in /etc/snort/snort.conf instead. This is useful if you are using Snort in a system which frequently changes network and does not have a static IP address assigned.

Please note that if Snort is configured to use multiple interfaces, it will use this value as the HOME_NET definition for all of them.

Address range for the local network:

10.0.1.0/24

<Ok>

Step 3: Verifying Snort Installation

Verified that Snort was installed correctly by checking:

- Version information
- Linked libraries
- Successful startup without errors

Purpose:

To confirm Snort is installed and operational.

```
root@client:/home/ubuntu# snort -V
```

```
''_      -*> Snort! <*-
o"_)~    Version 2.9.20 GRE (Build 82)
'''      By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
          Copyright (C) 2014-2022 Cisco and/or its affiliates. All rights reserved.
          Copyright (C) 1998-2013 Sourcefire, Inc., et al.
          Using libpcap version 1.10.4 (with TPACKET_V3)
          Using PCRE version: 8.39 2016-06-14
          Using ZLIB version: 1.3
```

```
root@client:/home/ubuntu#
```

Step 4: Creating Custom ICMP Detection Rules

Custom rules were added to detect:

- ICMP Echo Requests
- ICMP Echo Replies

These rules helped understand:

- Rule syntax
- Direction operators
- ICMP types

Purpose:

To learn how Snort detects basic network activity using custom rules.

```
GNU nano 7.2 /etc/snort/rules/local.rules *
# $Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $
# -----
# LOCAL RULES
# -----
# This file intentionally does not come with signatures.  Put your local
# additions here.
# Echo Request (outbound)
alert icmp $HOME_NET any -> any any ( msg:"ICMP Echo Request Detected"; itype:8; sid:1000003; rev:1;)

# Echo Reply (inbound)
alert icmp any any -> $HOME_NET any ( msg:"ICMP Echo Reply Detected"; itype:0; sid:1000002; rev:1;)
```

Step 5: Validating Snort Configuration

Snort configuration was validated in **test mode** to ensure:

- No syntax errors
- Variables were defined correctly
- Rule files loaded successfully

Purpose:

To prevent runtime failures and ensure configuration correctness.

```
Preprocessor Object: SF_DNS Version 1.1 <Build 4>
Preprocessor Object: SF_MODBUS Version 1.1 <Build 1>
Preprocessor Object: SF_REPUTATION Version 1.1 <Build 1>
Preprocessor Object: SF_SIP Version 1.1 <Build 1>
Preprocessor Object: SF_SSLPP Version 1.1 <Build 4>
Preprocessor Object: SF_SDF Version 1.1 <Build 1>
Preprocessor Object: SF_DNP3 Version 1.1 <Build 1>
Preprocessor Object: SF_FTPTELNET Version 1.2 <Build 13>

Total snort Fixed Memory Cost - MaxRss:105148
Snort successfully validated the configuration!
Snort exiting
root@client:/home/ubuntu#
```

Step 6: Generating ICMP Traffic for Testing

ICMP ping traffic was generated from a separate terminal to trigger the custom rules.

Purpose:

To simulate real network activity and test rule effectiveness.

```
root@client:/home/ubuntu# sudo nano /etc/snort/rules/local.rules
root@client:/home/ubuntu# ping -c 3 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=115 time=1.49 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=115 time=1.04 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=115 time=1.03 ms

--- 8.8.8.8 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 1.031/1.184/1.485/0.212 ms
root@client:/home/ubuntu#
```

Step 7: Observing Snort Alerts

Snort successfully generated alerts for:

- ICMP Echo Requests
- ICMP Echo Replies

Alerts showed:

- Source IP
- Destination IP
- Protocol type
- Alert message

Purpose:

To confirm Snort is actively inspecting traffic and generating detections.

```
Commencing packet processing (pid=3915)
02/06-18:37:21.023698  [**] [1:1000003:1] ICMP Echo Request Detected [**] [Priority: 0] {ICMP} 10.0.1.214 -> 8.8.8.8
02/06-18:37:21.025176  [**] [1:1000002:1] ICMP Echo Reply Detected [**] [Priority: 0] {ICMP} 8.8.8.8 -> 10.0.1.214
02/06-18:37:22.025450  [**] [1:1000003:1] ICMP Echo Request Detected [**] [Priority: 0] {ICMP} 10.0.1.214 -> 8.8.8.8
02/06-18:37:22.026470  [**] [1:1000002:1] ICMP Echo Reply Detected [**] [Priority: 0] {ICMP} 8.8.8.8 -> 10.0.1.214
02/06-18:37:23.026836  [**] [1:1000003:1] ICMP Echo Request Detected [**] [Priority: 0] {ICMP} 10.0.1.214 -> 8.8.8.8
02/06-18:37:23.027848  [**] [1:1000002:1] ICMP Echo Reply Detected [**] [Priority: 0] {ICMP} 8.8.8.8 -> 10.0.1.214
```

Step 8: Exploring Snorpy (GUI-Based Rule Creation)

Snorpy was used to:

- Create rules using a web interface
- Avoid manual syntax errors
- Easily define protocol, ports, and messages

An **FTP Authentication Attempt** rule was created using Snorpy.

Purpose:

To demonstrate alternate and efficient methods of rule creation.

The screenshot shows the SNORPY web interface, titled "A Web Based Snort Rule Creator / Maker for Building Simple Snort Rules". The interface is divided into several sections for configuring a rule.

Top Configuration Bar:

- Alert: ☐ alert
- Protocol: ☐ tcp
- Direction: ☐ any
- Port: ☐ any
- Destination: ☐ \$HOME_NET
- Port: ☐ 21
- Signature: ☐ 1000004
- Revision: ☐ 1

Rule Name: FTP Authentication Attempt

Class-Type: ☐ Class-Type

Priority: ☐ Priority

gid: ☐ gid

TCP Section:

- HTTP REQUEST METHO: ☐ HTTP REQUEST METHO
- HTTP RESPONSE CODE: ☐ HTTP RESPONSE CODE
- ACK: ☐ SYN: ☐ PSH: ☐ RST: ☐ FIN: ☐ URG: ☐ + ☐ * ☐
- DIRECTION: ☐ DIRECTION
- TCP STATE: ☐ TCP STATE

Data Size: ☐ Data Size

Reference: ☐ Reference

Threshold Tracking Type: ☐ Threshold Tracking Type

TRK BY: ☐ TRK BY

Count #: ☐ Count #

Seconds: ☐ Seconds

Add Content Match: ☐ Add Content Match

Add Regex Match: ☐ Add Regex Match

Generated Rule:

```
alert tcp any any -> $HOME_NET 21 ( msg:"FTP Authentication Attempt"; sid:1000004; rev:1; )
```

Step 9: Adding Snorpy-Generated Rules to Snort

The rule created in Snorpy was added to local.rules.

Purpose:

To integrate GUI-generated rules into a live Snort environment.

```

GNU nano 7.2 /etc/snort/rules/local.rules
# $Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $
# -----
# LOCAL RULES
# -----
# This file intentionally does not come with signatures.  Put your local
# additions here.
# Echo Request (outbound)
alert icmp $HOME_NET any -> any any ( msg:"ICMP Echo Request Detected"; itype:8; sid:1000003; rev:1;)

# Echo Reply (inbound)
alert icmp any any -> $HOME_NET any ( msg:"ICMP Echo Reply Detected"; itype:0; sid:1000002; rev:1;)

alert tcp any any -> $HOME_NET 21 ( msg:"FTP Authentication Attempt"; sid:1000004; rev:1; )

```

Step 10: Simulating FTP Attack from Kali Linux

From the Kali attacker machine, an FTP authentication attempt was made toward the Snort-monitored host.

Purpose:

To simulate attacker behavior and test TCP-based detection rules.

```

(kali㉿ kali)-[~]
$ ftp 10.10.10.21
ftp: Can't connect to 10.10.10.21:21: Connection refused
ftp: Can't connect to 10.10.10.21:21
ftp>

```

Step 11: Detecting Attacker Activity in Snort

Snort successfully generated an alert showing:

- Attacker IP (Kali)
- Target IP (Snort EC2)
- Protocol and destination port (FTP – 21)

Purpose:

To validate real attacker detection.

```

Preprocessor Object: SF_SIP Version 1.1 <Build 1>
Preprocessor Object: SF_SSLPP Version 1.1 <Build 4>
Preprocessor Object: SF_SDF Version 1.1 <Build 1>
Preprocessor Object: SF_DNP3 Version 1.1 <Build 1>
Preprocessor Object: SF_FTPTELNET Version 1.2 <Build 13>
Commencing packet processing (pid=5000)
02/06-19:13:46.318569  [**] [1:1000004:1] FTP Authentication Attempt [**] [Priority: 0] {TCP} 3.90.10.10:54988 -> 10.10.10.21

```

Step 12: Configuring Wazuh Agent to Monitor Snort Logs

The Snort fast alert log file was added to the Wazuh agent configuration using localfile.

Purpose:

To forward Snort alerts to Wazuh for centralized monitoring.

```
<localfile>
  <log_format>syslog</log_format>
  <location>/var/log/snort/snort.alert.fast</location>
</localfile>
```

Step 13: Correlating Snort Alerts in Terminal

Alerts seen in:

- /var/log/snort/snort.alert.fast
- Wazuh agent logs

Purpose:

To verify successful ingestion of Snort alerts in log file location for Wazuh.

```
root@client:/home/ubuntu# tail -f /var/log/snort/snort.alert.fast
02/06-18:41:09.982122  [**] [1:1917:6] SCAN UPnP service discover attempt [**] [Classification: Detection of a Network Scan] [Priority: 3] (UDP) 176.65.148.49:38787 -> 10.0.1.214:1900
02/06-18:44:54.661351  [**] [1:2049:4] MS-SQL ping attempt [**] [Classification: Misc activity] [Priority: 3] (UDP) 146.88.241.117:43033 -> 10.0.1.214:1900
02/06-19:30:42.611424  [**] [1:1867:1] MISC xdmcp info query [**] [Classification: Attempted Information Leak] [Priority: 2] (UDP) 205.210.31.92:34087 -> 10.0.1.214:1900
02/06-19:46:16.973490  [**] [1:1917:6] SCAN UPnP service discover attempt [**] [Classification: Detection of a Network Scan] [Priority: 3] (UDP) 8.211.52.116:58954 -> 10.0.1.214:1900
```

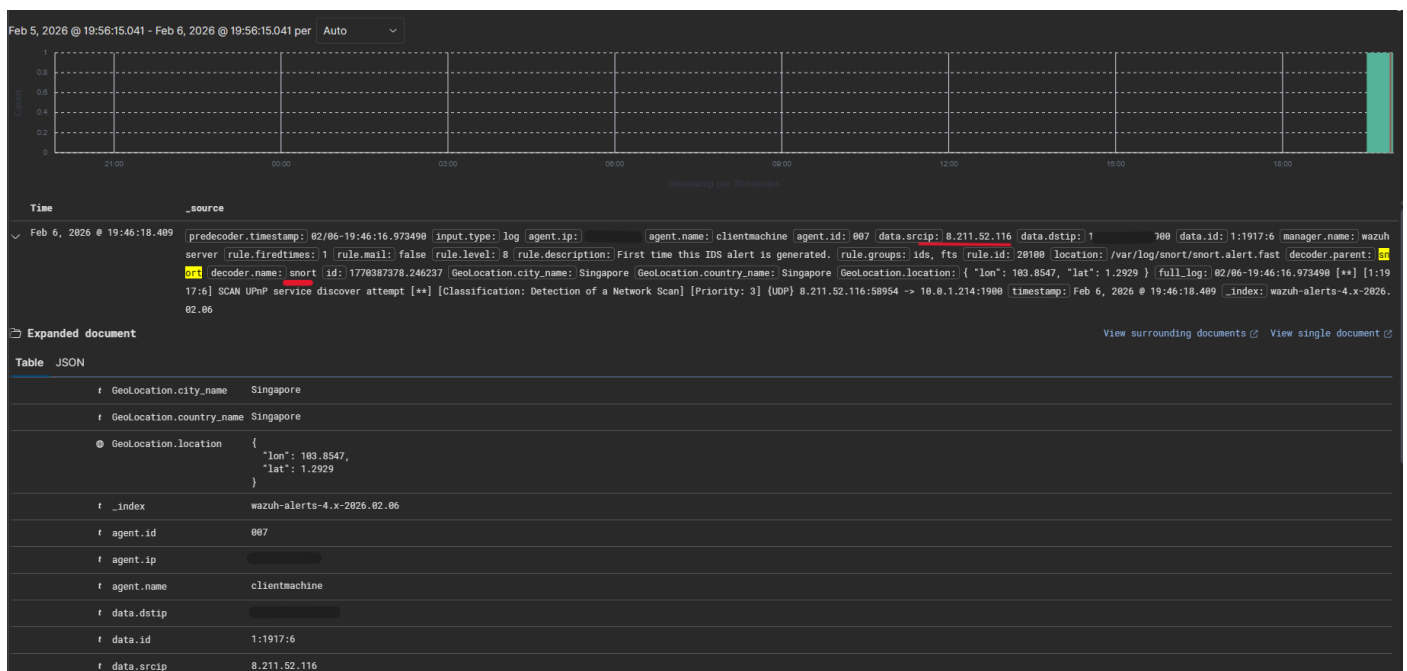
Step 14: Visualizing Snort Alerts in Wazuh SIEM

Snort alerts were visible in the Wazuh dashboard with:

- Full alert message
- Source and destination IPs
- Classification and severity
- Timestamp and log source

Purpose:

To demonstrate centralized network security monitoring using SIEM.



t decoder.parent	snort
t full_log	02/06-19:46:16.973490 [**] [1:1917:6] SCAN UPnP service discover attempt [**] [Classification: Detection of a Network Scan] [Priority: 3] {UDP} 8.211.52.116:58954 ->
t id	1770387378.246237
t input.type	log
t location	/var/log/snort/snort.alert.fast
t manager.name	wazuhserver
t predecoder.timestamp	02/06-19:46:16.973490
t rule.description	First time this IDS alert is generated.
# rule.firedtimes	1
t rule.groups	ids, fts
t rule.id	28100
# rule.level	8
rule.mail	false
timestamp	Feb 6, 2026 @ 19:46:18.489

Project Outcomes

- Successfully installed and configured Snort IDS
- Created and tested multiple custom detection rules
- Used Snorpy for GUI-based rule development
- Simulated attacker behavior from Kali Linux
- Integrated Snort with Wazuh SIEM
- Achieved centralized visibility of network threats

Skills Gained

- Network traffic analysis
- Snort rule writing and testing
- IDS deployment in cloud environments
- Attacker simulation techniques
- SIEM log ingestion and analysis
- SOC-level troubleshooting and validation

Conclusion

This project provided a deep, practical understanding of **network intrusion detection** using Snort and demonstrated how integrating IDS alerts into a SIEM like Wazuh enhances visibility, correlation, and incident response.

It reflects real-world SOC workflows where network and endpoint telemetry are combined to detect and investigate threats effectively.
